

Portfolio GTM Standardization Scorecard

How to measure, automate and hand off a standard go-to-market system across portfolio companies, proven on two companies with opposite motions.

1 · Executive summary

2 · Architecture and data model

3 · HubSpot configuration

4 · Attribution and financial logic

5 · AI lead agent

6 · Dashboard setup

7 · Rolling out to a new company

8 · GTM Engineer onboarding

Appendix · Runbook and assumptions

SCOPE

Acme MSP (inbound) · Summit HVAC
(outbound)

DATA

Synthetic raw exports, Apr-Sep 2026
(through 2026-09-30)

GENERATED

2026-10-04

1 Executive summary

Two portfolio companies sell in different ways: Acme MSP wins managed-IT contracts mostly from inbound search, and Summit HVAC wins commercial service work mostly through outbound and the phone. This system measures both on one set of definitions, automates lead handling, and documents everything a GTM Engineer needs to run it and roll it out.

ACME MSP · INBOUND-HEAVY

\$10,577	3.40x	14.1 mo	1.40x
Blended CAC	LTV : CAC	CAC payback	ROAS

SUMMIT HVAC · OUTBOUND-HEAVY

\$4,668	2.44x	14.8 mo	1.83x
Blended CAC	LTV : CAC	CAC payback	ROAS

What the scorecard shows

- **Acme MSP:** Referral is the cheapest channel at \$1,926 CAC (19.41x LTV:CAC). Outbound costs \$32,369 per customer and takes 24.5% of spend for 7.9% of bookings.
- **Summit HVAC:** Referral is the cheapest channel at \$1,343 CAC (10.20x LTV:CAC). Meta Ads costs \$14,935 per customer and takes 8.6% of spend for 6.4% of bookings.
- Median speed to lead is 3.6 h at Acme MSP and 6.0 h at Summit HVAC against a 5-minute SLA. The AI agent routes a lead and creates the SLA task in milliseconds.
- At Summit HVAC, 61.0% of bookings came from buyers who phoned in, yet 28.4% of tracked calls were missed. Answering the phone is the cheapest growth lever in the portfolio.
- Only 87.0% of records at Acme MSP and 75.7% of records at Summit HVAC carry every required field. Completeness is on the scorecard so it gets managed like any other KPI.

Build once, measure consistently, automate, document, propagate. Everything company-specific lives in one config file per company. The data model, metric definitions, HubSpot schema, agent logic and dashboard are shared. A new company needs its data exports and its pipeline names, nothing else (Section 7).

Portfolio scorecard

METRIC	ACME MSP	SUMMIT HVAC	TARGET	DEFINITION
Leads	3,006	1,779		Leads created in period
Opportunities	216	203		Deals that reached Qualified
Closed-won deals	50	74		Deals in Won stage

Metric	Acme MSP	Summit HVAC	Target	Definition
Bookings (closed-won revenue)	\$741,450	\$630,850		Sum of won deal amount (first-year contract value)
Open pipeline	\$1,153,650	\$513,350		Amount on deals not yet Won/Lost
GTM spend	\$528,866	\$345,463		Media + outbound + referral + call-tracking spend
Lead to opportunity	7.2%	11.4%	8.0%	Opportunities / leads
Opportunity to won	27.0%	38.5%	30.0%	Won / closed opportunities (Won + Lost after Qualified)
Speed to lead (median)	3.6 h	6.0 h	15 min	Lead created to first rep activity
Average contract value	\$14,829	\$8,525		Mean amount of won deals
Gross margin	60.6%	44.5%		Revenue-weighted margin on won deals
Blended CAC	\$10,577	\$4,668		Total GTM spend / closed-won deals
ROAS	1.40x	1.83x	1.50x	Bookings / GTM spend
Customer lifetime (assumption)	4.0 yrs	3.0 yrs		Set per company in config (from churn)
LTV	\$35,942	\$11,370		ACV x gross margin x customer lifetime
LTV : CAC	3.40x	2.44x	3.00x	LTV / blended CAC
CAC payback	14.1 mo	14.8 mo	12.0 mo	CAC / (ACV x gross margin / 12)
Sales cycle	39.3 d	20.7 d		Deal created to Won, mean
Pipeline velocity	\$22,019/day	\$32,263/day		Opps x win rate x ACV / sales cycle days
Data completeness	87.0%	75.7%	90.0%	Share of lead + deal records with every required field populated
Inbound calls tracked	873	1,042		Calls logged by call tracking
Missed call rate	13.5%	28.4%	10.0%	Missed / tracked calls
Call-influenced bookings	33.3%	61.0%		Share of bookings where the buyer phoned in at least once
Leads processed by agent	564	260		Leads run through the AI agent end to end
Manual steps eliminated	3,948	1,820		Automated steps x leads
Rep hours saved	159.8 h	73.7 h		Steps x manual minutes per step
Routing time (agent, median)	0.4 ms	0.4 ms		Webhook received to deal + task created

Targets come from `config/standard.json`. Lead-to-opportunity and win rate are realized figures; deals still open at the data cut-off are not yet counted as won or lost.

2 Architecture and data model

Five layers, each replaceable without touching the others. Synthetic data stands in for real exports; mocks stand in for live HubSpot and Slack until credentials are set.

Data sources	CRM exports, ad platforms, call tracking. In the POC, <code>gtm/generate.py</code> writes the same CSV schemas from the per-company config.		
HubSpot (system of record)	Standard custom properties, one pipeline per motion, routing, lifecycle and SLA workflows. Schema generated by <code>gtm/hubspot.py</code> .		
AI lead agent	Webhook on new lead → enrich, score, route, create deal, SLA task, Slack, log.	Attribution and finance	SQLite + pandas: funnel, multi-touch attribution, CAC, ROAS, LTV, payback, velocity.
Executive dashboard	Self-contained HTML build plus a Looker Studio version, both reading <code>output/*.csv</code> .		
Playbook	This document, generated from the same config and outputs so it cannot drift.		

Standard data model

FILE	GRAIN	KEY COLUMNS
leads.csv	One row per lead	lead_id, company, contact_name, title, email, created_date, lead_source, utm_source/medium/campaign, territory, fit_score
deals.csv	One row per deal	deal_id, lead_id, stage, stage_reached, amount, close_date, lead_source, gross_margin, pipeline, owner, created_date
spend.csv	Channel × month	channel, month, spend, impressions, clicks
activities.csv	One rep activity	activity_id, lead_id, type (call/email/meeting), date, outcome, owner
call_tracking.csv	One tracked call	call_id, lead_id, source, channel, call_datetime, duration, outcome, revenue_influenced
touchpoints.csv	One marketing touch	touch_id, lead_id, touch_datetime, channel, utm_campaign, touch_type
deal_stage_history.csv	Deal × stage	deal_id, stage, entered_at, exited_at

`touchpoints` and `deal_stage_history` extend the original design: multi-touch attribution needs every touch, not just the lead source, and days-in-stage needs stage timestamps. `stage_reached` on deals records the furthest stage a deal got to, which the opportunity rule depends on.

Cleaning and normalization layer

Raw exports never match each other. The same channel arrives as "AdWords", "google_cpc" or "Google Ads Search"; margins come as 0.61, 61 or "61%"; stages as "Closed Won" or "closedwon". `gtm/clean.py` fixes all of it before any metric is computed, driven by `channel_aliases`, `stage_aliases` and `state_names` in `config/standard.json`, and logs every fix. A round-trip test generates clean data, makes it messy, cleans it and checks the result matches exactly.

TABLE	FIELD	FIX	ACME MSP	SUMMIT HVAC
activities	type	Activity type lower-cased	775	1012
call_tracking	channel	Source name mapped to standard channel	168	358
deals	amount	Currency text parsed to number	54	124
deals	gross_margin	Percent-style margin converted to fraction	108	223
deals	lead_source	Source name mapped to standard channel	107	252
deals	stage	Stage label mapped to standard stage	115	243
leads	created_date	Non-ISO date format parsed	291	302
leads	email	Duplicate lead removed (same email, later record)	60	36
leads	email	Email trimmed / lower-cased	255	270
leads	lead_source	Source name mapped to standard channel	551	614
leads	territory	Territory normalized (case / state name)	239	297
leads	utm_campaign	UTM value trimmed / lower-cased / spaces to underscores	169	127
leads	utm_source	UTM value trimmed / lower-cased / spaces to underscores	197	151
spend	channel	Source name mapped to standard channel	30	30
spend	spend	Currency text parsed to number	30	30
touchpoints	channel	Source name mapped to standard channel	1458	1508

Normalization then puts both companies on equal terms in four steps: **clean** onto one vocabulary; apply one set of **definitions**; aggregate to a common **grain** (company × month × channel) so any date range or channel subset compares fairly; and **index to target** (100 = on target) so metrics with different units sit on one scale.

Portfolio standard

Channels: Google Ads, Meta Ads, Outbound, Referral, Call Tracking. Stages in every pipeline: New → Contacted → Qualified → Proposal → Won → Lost. An **opportunity** is any deal that has reached Qualified, including deals later lost.

3 HubSpot configuration

Everything runs on the **free HubSpot CRM**. One portal holds every company: `Company Type` says which company owns a record, and `GTM Motion` says which of its pipelines a deal belongs to. Free allows one deal pipeline and no workflows, so the AI agent does the routing and SLA work that workflows would do. Custom properties use a `gtm_` prefix so they never collide with HubSpot defaults; labels match the import files so columns auto-map.

Custom properties

LABEL	INTERNAL NAME	OBJECTS	TYPE	OPTIONS / NOTES
Lead Source	gtm_lead_source	contacts, deals	select	Google Ads; Meta Ads; Outbound; Referral; Call Tracking
UTM Source	gtm_utm_source	contacts	text	utm_source of the converting session
UTM Medium	gtm_utm_medium	contacts	text	utm_medium of the converting session
UTM Campaign	gtm_utm_campaign	contacts	text	e.g. msp_q3_search
Company Type	gtm_company_type	contacts, deals	select	HVAC; MSP
Territory	gtm_territory	contacts, deals	select	AZ; CA; FL; GA; IL; MA; NC; NJ; NY; TX; Unassigned
Fit Score	gtm_fit_score	contacts, deals	number	0-100, set by the AI agent
Gross Margin	gtm_gross_margin	deals	number	Expected gross margin, percent (0-100)
Service Type	gtm_service_type	contacts, deals	select	Emergency; Installation; Maintenance
Lead Created Date	gtm_lead_created_date	contacts	date	Original lead creation time (kept separate from HubSpot createdate on import)
Assigned Rep	gtm_assigned_rep	contacts, deals	text	Rep name from territory routing; map to HubSpot owners once users exist
GTM Motion	gtm_motion	deals	select	Inbound; Outbound; Referral; Emergency; Maintenance; Installation

Pipeline and motions

One HubSpot pipeline, **GTM Pipeline**, with stages New → Contacted → Qualified → Proposal → Won → Lost. Each company's logical pipelines from config become values of `GTM Motion` ; saved board views (Company Type + GTM Motion) give each team its own board.

COMPANY	LOGICAL PIPELINE (CONFIG)	HUBSPOT PIPELINE	GTM MOTION	ROUTING RULE
Acme MSP	Acme MSP -Inbound	GTM Pipeline	Inbound	Lead source: Google Ads, Meta Ads, Call Tracking
Acme MSP	Acme MSP -Outbound	GTM Pipeline	Outbound	Lead source: Outbound
Acme MSP	Acme MSP -Referral	GTM Pipeline	Referral	Lead source: Referral
Summit HVAC	Summit HVAC -Emergency	GTM Pipeline	Emergency	Service type: Emergency
Summit HVAC	Summit HVAC -Maintenance	GTM Pipeline	Maintenance	Service type: Maintenance
Summit HVAC	Summit HVAC -Installation	GTM Pipeline	Installation	Service type: Installation

Territories and lead routing

COMPANY	TERRITORY	STATES	OWNER
Acme MSP	NY	NY	Dana Whitfield
Acme MSP	NJ	NJ, PA	Marcus Lee
Acme MSP	MA	MA, CT, RI	Priya Raman
Acme MSP	TX	TX	Jordan Ellis
Acme MSP	CA	CA	Sofia Alvarez
Acme MSP	IL	IL	Owen Brooks
Acme MSP	FL	FL	Grace Kim
Summit HVAC	TX	TX, OK	Caleb Turner
Summit HVAC	AZ	AZ, NM, NV	Maya Ortiz
Summit HVAC	GA	GA, SC	Andre Coleman
Summit HVAC	FL	FL	Hannah Price
Summit HVAC	NC	NC, VA	Ben Foster
Summit HVAC	CA	CA	Lucia Romero

States outside every territory route to the company's unassigned queue (round robin or dispatch).

Automation without workflows

NEED	FREE CRM (DEFAULT)	PAID UPGRADE (OPTIONAL)
Lead routing	Agent sets Territory and Assigned Rep from state	Territory routing workflow
SLA	Agent creates a 5-minute follow-up task on every lead	SLA workflow with escalation

NEED	FREE CRM (DEFAULT)	PAID UPGRADE (OPTIONAL)
Lifecycle stage	Set on import from deal status	Workflow on deal-stage change
Pipelines	One pipeline + GTM Motion	Six pipelines: set <code>hubspot.tier</code> to "pro"

Setup steps

1. Create a private app (contacts and deals read/write, schema write). Put the token in `.env` as `HUBSPOT_TOKEN`.
2. `python -m gtm.hubspot setup --apply` creates the property group and properties and reshapes the default pipeline into GTM Pipeline. It is safe to re-run.
3. Import per company: contacts; deals with contacts (matched on Email); then calls, meetings and emails-as-notes activity files.
4. Save one deal board view per Company Type + GTM Motion.
5. Subscribe the private app's webhook to `contact.creation`, pointing at the agent's `/webhooks/hubspot`.

4 Attribution and financial logic

Every metric has one definition, written once in code and applied to every company. The definitions below are the contract; if a company reports a number that doesn't match them, it isn't the scorecard number.

CATEGORY	METRIC	STANDARD DEFINITION
Volume	Leads	Leads created in period
Volume	Opportunities	Deals that reached Qualified
Volume	Closed-won deals	Deals in Won stage
Volume	Bookings (closed-won revenue)	Sum of won deal amount (first-year contract value)
Volume	Open pipeline	Amount on deals not yet Won/Lost
Volume	GTM spend	Media + outbound + referral + call-tracking spend
Conversion	Lead to opportunity	Opportunities / leads
Conversion	Opportunity to won	Won / closed opportunities (Won + Lost after Qualified)
Conversion	Speed to lead (median)	Lead created to first rep activity
Economics	Average contract value	Mean amount of won deals
Economics	Gross margin	Revenue-weighted margin on won deals
Economics	Blended CAC	Total GTM spend / closed-won deals
Economics	ROAS	Bookings / GTM spend
Economics	Customer lifetime (assumption)	Set per company in config (from churn)
Economics	LTV	ACV x gross margin x customer lifetime
Economics	LTV : CAC	LTV / blended CAC
Economics	CAC payback	CAC / (ACV x gross margin / 12)
Velocity	Sales cycle	Deal created to Won, mean
Velocity	Pipeline velocity	Opps x win rate x ACV / sales cycle days
Data & Ops	Data completeness	Share of lead + deal records with every required field populated
Data & Ops	Inbound calls tracked	Calls logged by call tracking
Data & Ops	Missed call rate	Missed / tracked calls
Data & Ops	Call-influenced bookings	Share of bookings where the buyer phoned in at least once
AI Agent	Leads processed by agent	Leads run through the AI agent end to end
AI Agent	Manual steps eliminated	Automated steps x leads
AI Agent	Rep hours saved	Steps x manual minutes per step

CATEGORY	METRIC	STANDARD DEFINITION
AI Agent	Routing time (agent, median)	Webhook received to deal + task created

Attribution models

- **First-touch:** 100% of a won deal's amount to the first touch. Answers "what creates demand?"
- **Last-touch:** 100% to the last touch before the deal was created. Answers "what converts demand?"
- **Linear:** equal split across every touch up to deal creation. The default for budget discussions, because no single touch gets all the credit.

Design decisions. (1) Opportunities count deals that reached Qualified, even if later lost; filtering on the current stage undercounts opportunities and inflates win rate. (2) CAC includes the fully loaded outbound team cost, referral payouts and call-tracking fees, not only media, so channels compare fairly. (3) Pipeline velocity uses the standard formula (opportunities × win rate × ACV ÷ cycle days), in dollars per day. (4) Customer lifetime is a per-company assumption from churn, set in config and shown on the scorecard.

sql/channel_funnel.sql

```
-- Standard channel funnel: spend → leads → opportunities → won → revenue, per portfolio company.
-- Corrections vs the design-doc pseudo-SQL:
-- 1. Opportunities use stage_reached, so deals that were qualified and later Lost still count
--    (filtering on current stage IN ('Qualified','Proposal','Won') undercounts opps and inflates win rate).
-- 2. Gross margin is averaged inside the won CTE; joining deals again in the final SELECT
--    fans out the rows and multiplies spend.
-- 3. Every CTE is keyed by portfolio_company so one query serves the whole portfolio.
WITH spend_c AS (
    SELECT portfolio_company, channel, SUM(spend) AS total_spend
    FROM spend
    GROUP BY portfolio_company, channel
),
leads_c AS (
    SELECT portfolio_company, lead_source AS channel, COUNT(*) AS leads
    FROM leads
    GROUP BY portfolio_company, lead_source
),
opps_c AS (
    SELECT portfolio_company, lead_source AS channel, COUNT(*) AS opps,
           SUM(CASE WHEN stage IN ('Won', 'Lost') THEN 1 ELSE 0 END) AS closed_opps
    FROM deals
    WHERE stage_reached IN ('Qualified', 'Proposal', 'Won')
    GROUP BY portfolio_company, lead_source
),
won_c AS (
    SELECT portfolio_company, lead_source AS channel, COUNT(*) AS won, SUM(amount) AS revenue,
           SUM(amount * gross_margin) / NULLIF(SUM(CASE WHEN gross_margin IS NOT NULL THEN amount END), 0) AS
    avg_margin
    FROM deals
    WHERE stage = 'Won'
    GROUP BY portfolio_company, lead_source
)
SELECT
    s.portfolio_company,
    s.channel,
    s.total_spend,
    COALESCE(l.leads, 0) AS leads,
    COALESCE(o.opps, 0) AS opps,
    COALESCE(o.closed_opps, 0) AS closed_opps,
```

```

        COALESCE(w.won, 0)          AS won,
        COALESCE(w.revenue, 0)      AS revenue,
        s.total_spend / NULLIF(w.won, 0) AS cac,
        w.revenue / NULLIF(s.total_spend, 0) AS roas,
        w.avg_margin
FROM spend_c s
LEFT JOIN leads_c l ON l.portfolio_company = s.portfolio_company AND l.channel = s.channel
LEFT JOIN opps_c o ON o.portfolio_company = s.portfolio_company AND o.channel = s.channel
LEFT JOIN won_c w ON w.portfolio_company = s.portfolio_company AND w.channel = s.channel
ORDER BY s.portfolio_company, s.total_spend DESC;

```

sql/attribution.sql

```

-- Multi-touch attribution of closed-won revenue. A won deal's journey is every touch on its lead
-- up to the moment the deal was created. Each model splits the deal amount across those touches.
-- Grain: company x close month x channel, so dashboards can filter by date.
WITH journey AS (
    SELECT d.portfolio_company, substr(d.close_date, 1, 7) AS close_month, d.deal_id, d.amount,
           t.channel, t.touch_datetime,
           ROW_NUMBER() OVER (PARTITION BY d.deal_id ORDER BY t.touch_datetime ASC) AS pos_first,
           ROW_NUMBER() OVER (PARTITION BY d.deal_id ORDER BY t.touch_datetime DESC) AS pos_last,
           COUNT(*) OVER (PARTITION BY d.deal_id) AS n_touches
    FROM deals d
    JOIN touchpoints t
      ON t.portfolio_company = d.portfolio_company AND t.lead_id = d.lead_id
      AND t.touch_datetime ≤ d.created_date
    WHERE d.stage = 'Won'
)
SELECT portfolio_company, close_month, channel,
       SUM(CASE WHEN pos_first = 1 THEN amount ELSE 0 END) AS first_touch_revenue,
       SUM(CASE WHEN pos_last = 1 THEN amount ELSE 0 END) AS last_touch_revenue,
       SUM(amount * 1.0 / n_touches) AS linear_revenue,
       COUNT(DISTINCT deal_id) AS deals_touched
FROM journey
GROUP BY portfolio_company, close_month, channel
ORDER BY portfolio_company, close_month, channel;

```

Customer economics in Python

gtm/model.py

```

cac = safe_div(spend_total, len(W))
ltv = acv_ * gm * cfg["economics"]["customer_lifetime_years"]
cycle = (W.close_date - W.created_date).dt.total_seconds().mean() / 86400
win_rate = safe_div(len(W), len(closed_opps))
velocity = len(opps) * win_rate * acv_ / cycle

```

5 AI lead agent

The agent handles a new lead end to end with no manual step. DeepSeek (`deepseek-chat` , JSON mode, OpenAI-compatible API) scores fit against the company's ICP, writes the reason the rep reads and proposes a territory. A rules-based score is computed first and sent as the baseline; it becomes the score whenever there is no key or the model's answer is malformed, so the agent never blocks. Config stays the source of truth for territory: a model territory that disagrees with the state mapping is overridden.

1 · Enrich Firmographics by company name (mock provider; swap for Apollo or Clearbit)	2 · Score DeepSeek 0–100 fit score and reason; rules score as baseline and fallback	3 · Route State → territory → owner, from config	4 · Create deal In the right pipeline, stage New
5 · SLA task Follow up within 5 minutes	6 · Alert Slack message to the company's channel; HOT at 80+	7 · Log Activity note on the contact timeline	Trigger HubSpot <code>contact.creation</code> webhook (signature-verified)

Key: `DEEPSEEK_API_KEY` in `.env` (git-ignored; copy `.env.example`). Bulk replays cap model calls (`--llm-limit` , default 25) to keep cost near zero. **Rules baseline weights:** employees 30, industry 30, territory 20, source quality 20. **Source quality:** Referral 1.0, Call Tracking 0.85, Google Ads 0.7, Outbound 0.5, Meta Ads 0.4. Weights live in `config/standard.json` ; each company's ICP (industries, employee band) lives in its own config.

Impact, last 30 days replayed

COMPANY	LEADS PROCESSED (30 D)	STEPS AUTOMATED	REP HOURS SAVED	SPEED TO LEAD BEFORE	WITH AGENT	HOT LEADS
Acme MSP	564	3,948	160	3.6 h	5 min SLA	244
Summit HVAC	260	1,820	74	6.0 h	5 min SLA	100
STEP		MANUAL MINUTES REPLACED				
Enrich company		6				
Score fit		3				
Assign territory		2				
Create deal		3				
Create sla task		1				

Notify rep	1
Log activity	1

Integrations

VARIABLE	WHEN SET	WHEN NOT SET
HUBSPOT_TOKEN	Reads contacts; writes deals, tasks, notes through CRM v3	In-memory mock CRM seeded from leads.csv
HUBSPOT_CLIENT_SECRET	Rejects webhooks without a valid v3 signature	Accepts all webhooks (local demo)
SLACK_WEBHOOK_URL	Posts alerts to Slack	Writes alerts to output/slack_outbox.jsonl
DEEPSEEK_API_KEY	DeepSeek (deepseek-chat, JSON mode) scores fit 0-100, writes the rationale and proposes a territory	Rules-based score and template rationale

gtm/integrations.py

```
def deepseek_score(lead, enriched, cfg, rule_score, components):
    """Ask DeepSeek (deepseek-chat, JSON mode) for a fit score, reason and territory.

    Returns {"fit_score": int, "reason": str, "territory": str|None} or None when no key is set,
    the call fails, or the answer is malformed, so the agent falls back to the rules score and
    never blocks on the model. The territory is validated by the caller against config.
    """
    key = os.environ.get("DEEPSEEK_API_KEY")
    if not key:
        return None
    base = os.environ.get("DEEPSEEK_BASE_URL", "https://api.deepseek.com").rstrip("/")
    territories = {t: s["states"] for t, s in cfg["territories"].items()}
    system = ("You are a GTM operations agent for a private-equity portfolio company. "
              "Score inbound leads against the ideal customer profile. Return JSON only.")
    user = (
        f"Company: {cfg['name']} ({cfg['company_type']}, {cfg['gtm_motion']}).\n"
        f"ICP: industries {cfg['icp']['industries']}, {cfg['icp']['employees_min']}-{cfg['icp']['employees_max']}"
        f"employees.\n"
        f"Territories (territory → states): {json.dumps(territories)}. Use null if the state is in none.\n"
        f"Lead: {json.dumps({k: lead.get(k) for k in ('contact_name', 'title', 'company', 'lead_source',
        'utm_campaign')}})\n"
        f"Enriched: {json.dumps({k: enriched[k] for k in ('industry', 'employees', 'annual_revenue',
        'state')}})\n"
        f"Rules-based baseline score: {rule_score} (components 0-1: {json.dumps({k: round(v, 2) for k, v in
        components.items()})}).\n"
        'Score this lead 0-100 on ICP fit. Return json: {"fit_score": int, "reason": str (max 30 words, '
        'what the rep should lead with), "territory": str or null}'
    )
    try:
        res = _http("POST", f"{base}/chat/completions", key, {
            "model": os.environ.get("DEEPSEEK_MODEL", "deepseek-chat"),
            "messages": [{"role": "system", "content": system}, {"role": "user", "content": user}],
            "response_format": {"type": "json-object"},
            "temperature": 0.2, "max_tokens": 200,
        }, timeout=20)
        out = json.loads(res["choices"][0]["message"]["content"])
        score = int(round(float(out["fit_score"])))
        if not 0 ≤ score ≤ 100:
            return None
        return {"fit_score": score, "reason": str(out.get("reason") or "").strip(), "territory":
        out.get("territory")}
```

```
except (urllib.error.URLError, KeyError, IndexError, TimeoutError, ValueError, TypeError):
    return None
```

gtm/agent.py

```
"""AI lead agent: enrich → score → route → create deal → SLA task → Slack alert → log activity.

Runs with no accounts by default (MockHubSpot, MockSlack, rules-based score). Set HUBSPOT_TOKEN,
SLACK_WEBHOOK_URL and/or DEEPSEEK_API_KEY (in .env or the environment) to switch each
integration to the live service.
Everything company-specific (ICP, territories, owners, pipelines, Slack channel) comes from config.
"""
import os
import time
from datetime import datetime, timedelta

from .config import load_companies, load_standard, pipeline_for, territory_for_state
from .enrich import mock_enrich
from .integrations import deepseek_score
from .scoring import score_fit

def company_for_lead(lead, companies):
    """Pick the portfolio company config for a lead (by explicit slug, or by Company Type property)."""
    for cfg in companies:
        if lead.get("company_slug") == cfg["slug"] or lead.get("company_type") == cfg["company_type"]:
            return cfg
    raise ValueError(f"No portfolio company config matches lead {lead.get('lead_id')}")

def template_rationale(fit_score, comps, enriched, lead_source):
    strengths = [k.replace("_", " ") for k, v in comps.items() if v ≥ 0.8]
    gaps = [k.replace("_", " ") for k, v in comps.items() if v < 0.5]
    s = f"Fit {fit_score}/100: {enriched['industry']}, {enriched['employees']} employees, {enriched['state']}, via {lead_source}."
    if strengths:
        s += f" Strong on {'', '.join(strengths)}."
    if gaps:
        s += f" Weak on {'', '.join(gaps)}."
    return s

class LeadAgent:
    def __init__(self, crm, slack, companies=None, standard=None, use_llm=True):
        self.crm, self.slack = crm, slack
        self.companies = companies or load_companies()
        self.standard = standard or load_standard()
        self.use_llm = use_llm and bool(os.environ.get("DEEPSEEK_API_KEY"))

    def process_lead(self, lead, now=None):
        """Process one new lead end to end. `now` lets a replay use the lead's creation time."""
        t0 = time.perf_counter()
        now = now or datetime.now()
        cfg = company_for_lead(lead, self.companies)
        std = self.standard["agent"]
        contact_id = lead["lead_id"]

        # 1. Enrich company (mock provider; swap for Apollo/Clearbit in production)
        enriched = mock_enrich(lead["company"], cfg)
        if lead.get("state"):
            enriched["state"] = lead["state"]

        # 2. Score fit: DeepSeek scores against the ICP; the rules score is the baseline and the fallback
        rule_score, comps = score_fit(enriched, lead["lead_source"], cfg, self.standard)
        llm = deepseek_score(lead, enriched, cfg, rule_score, comps) if self.use_llm else None
        fit_score = llm["fit_score"] if llm else rule_score
        score_source = "deepseek" if llm else "rules"
        rationale = (llm or {}).get("reason") or template_rationale(fit_score, comps, enriched,
```

```

lead["lead_source"])
    hot = fit_score ≥ std["hot_lead_threshold"]

    # 3. Assign territory: config is the source of truth; an LLM territory is kept only if it agrees
    territory, owner = territory_for_state(cfg, enriched["state"])
    territory_source = "rules"
    if llm and llm.get("territory") in cfg["territories"]:
        territory_source = "deepseek" if llm["territory"] == territory else "rules (LLM overridden)"
    self.crm.update_contact(contact_id, {"fit_score": fit_score, "territory": territory or "Unassigned",
                                         "company_type": cfg["company_type"]})

    # 4. Create deal in the right pipeline
    service = lead.get("service_type") or cfg.get("default_service_by_lead_source",
    {}).get(lead["lead_source"])
    pipeline = pipeline_for(cfg, lead["lead_source"], service)
    deal = self.crm.create_deal(contact_id, {
        "dealname": f"{lead['company']} - {pipeline.split(' - ')[-1]}",
        "pipeline": pipeline, "dealstage": "New", "owner": owner,
        "lead_source": lead["lead_source"], "fit_score": fit_score, "territory": territory or "Unassigned",
    })

    # 5. SLA task: follow up within the portfolio SLA
    sla = std["sla_minutes"]
    task = self.crm.create_task(deal["id"], owner, f"Follow up within {sla} min: {lead['company']}",
                                due_at=now + timedelta(minutes=sla), body=rationale)

    # 6. Slack alert to the company's lead channel
    flame = "HOT " if hot else ""
    self.slack.send(cfg["slack_channel"],
                    f"{flame}New lead: {lead['company']} | Fit: {fit_score} | {lead['lead_source']} | "
                    f"{territory or 'Unassigned'} → {owner} | SLA {sla} min")

    # 7. Log activity on the contact timeline
    self.crm.create_note(contact_id, f"AI Agent processed lead. {rationale}", timestamp=now)

    steps = std["manual_minutes_per_step"]
    return {
        "portfolio_company": cfg["name"], "lead_id": contact_id, "company": lead["company"],
        "lead_source": lead["lead_source"], "fit_score": fit_score, "hot_lead": hot,
        "territory": territory or "Unassigned", "owner": owner, "pipeline": pipeline,
        "deal_id": deal["id"], "task_id": task["id"], "task_due": (now + timedelta(minutes=sla)).isoformat(),
        "rules_fit_score": rule_score, "score_source": score_source, "territory_source": territory_source,
        "rationale": rationale,
        "steps_automated": len(steps), "minutes_saved": sum(steps.values()),
        "processing_seconds": round(time.perf_counter() - t0, 4),
    }

def build_agent():
    """Pick live or mock integrations based on which credentials are set."""
    from .integrations import HubSpotClient, SlackWebhook
    from .mocks import MockHubSpot, MockSlack
    if os.environ.get("HUBSPOT_TOKEN"):
        crm = HubSpotClient()
    else:
        crm = MockHubSpot()
        for cfg in load_companies():
            crm.seed_contacts_from_csv(cfg["slug"])
    slack = SlackWebhook() if os.environ.get("SLACK_WEBHOOK_URL") else MockSlack()
    return LeadAgent(crm, slack)

```

gtm/scoring.py

```

"""Rule-based fit score (0-100): weighted sum of employee fit, industry match, served
territory and source quality. Weights live in config/standard.json so every portfolio
company is scored the same way; only the ICP (in the company config) differs."""
import math

```

```

from .config import territory_for_state

def employee_fit(employees, lo, hi):
    if lo ≤ employees ≤ hi:
        return 1.0
    edge = lo if employees < lo else hi
    distance = abs(math.log(max(employees, 1) / edge)) # log-distance outside the band
    return max(0.0, 1.0 - 0.9 * distance)

def score_components(enriched, lead_source, cfg, standard):
    icp = cfg["icp"]
    sq = standard["agent"]["source_quality"]
    territory, _ = territory_for_state(cfg, enriched["state"])
    return {
        "employees": employee_fit(enriched["employees"], icp["employees_min"], icp["employees_max"]),
        "industry": 1.0 if enriched["industry"] in icp["industries"] else 0.1,
        "territory": 1.0 if territory else 0.0,
        "source_quality": sq.get(lead_source, 0.5),
    }

def score_fit(enriched, lead_source, cfg, standard):
    weights = standard["agent"]["fit_weights"]
    comps = score_components(enriched, lead_source, cfg, standard)
    score = sum(weights[k] * comps[k] for k in weights)
    return int(round(min(100, max(0, score)))), comps

```

gtm/agent_server.py

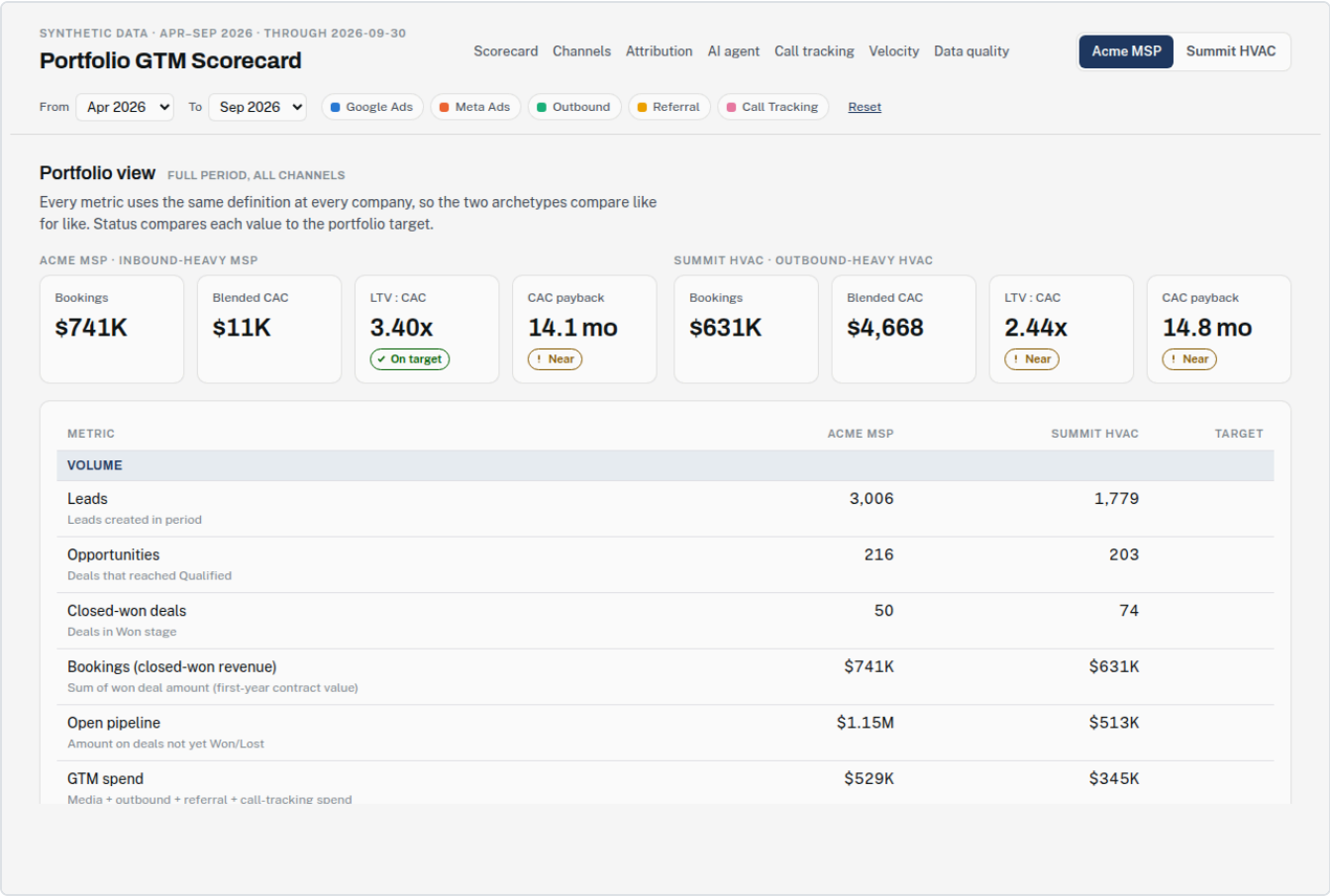
```

@app.post("/webhooks/hubspot")
def hubspot_webhook():
    verify_hubspot_signature()
    results = []
    for event in request.get_json(force=True) or []:
        if event.get("subscriptionType") ≠ "contact.creation":
            continue
        contact = agent.crm.get_contact(event["objectId"])
        if not contact:
            results.append({"objectId": event["objectId"], "error": "contact not found"})
            continue
        contact.setdefault("lead_id", str(event["objectId"]))
        results.append(agent.process_lead(contact))
    _persist()
    return jsonify(results)

```


6 Dashboard setup

Two builds read the same model outputs, filtered by company, channel and date. The HTML dashboard regenerates on every pipeline run. The Looker Studio version reads a Google Sheet that `gtm/sheets_sync.py` refreshes, so a scheduled run keeps it current with no manual step.



PAGE	SOURCE TAB	CHARTS
Portfolio view	scorecard	Pivot: metric × company with targets and status; KPI tiles for CAC, LTV:CAC, payback, bookings
Channel performance	channel_performance	Bars for CAC, ROAS, LTV:CAC by channel (3.0x reference line); funnel table
Attribution comparison	attribution_long	100% stacked bar: model × channel; ROAS by model table
AI automation impact	ai_automation_impact, fit_score_validation, agent_runs	KPI tiles; conversion by fit band; processed-lead table
Call-tracking impact	call_tracking_impact	Calls, booked and missed rate, revenue influenced by source

PAGE	SOURCE TAB	CHARTS
Velocity and quality	stage_days, monthly_trend, data_completeness	Days in stage pivot; monthly leads and bookings; field completeness

1. Create a Google Sheet; load each `output/*.csv` as its own tab (File → Import, or `python -m gtm.sheets_sync`).
2. In Looker Studio add each tab as a Google Sheets data source.
3. Add a company drop-down control on `portfolio_company` ; fix channel colors with Dimension value colors so a channel is the same color on every chart.
4. Build the pages above. Share view-only, or download as PDF for board packs.

7 Rolling out to a new company

The success test: a new company needs only its data inputs and its pipeline names. This was checked by adding a third test company with nothing but a new config file; the scorecard, dashboard, HubSpot files and agent routing all picked it up with no code change.

1. **Config.** Copy a company file to `config/companies/<slug>.json`; set pipelines and their routing rule, territories and owners, ICP, customer lifetime and Slack channel.
2. **Data.** Drop the seven raw exports into `data/<slug>/raw/`. The cleaning step normalizes them; add any source name it reports as unknown to `channel_aliases`.
3. **HubSpot.** Run `python -m gtm.hubspot setup --apply` to add the new dropdown values, then import.
4. **Run.** `python run_all.py --skip-generate`. Review the data-completeness section first; fix gaps at the source before trusting CAC.
5. **Agent.** Point the company's HubSpot webhook at the agent; watch the first 20 leads in Slack.

Company config (example, demo-data block omitted)

```
{
  "slug": "acme_msp",
  "name": "Acme MSP",
  "company_type": "MSP",
  "gtm_motion": "Inbound-heavy",
  "slack_channel": "#acme-new-leads",
  "unassigned_owner": "Acme Round Robin",
  "economics": {"target_acv": 15000, "target_gross_margin": 0.6, "customer_lifetime_years": 4.0, "lifetime_note": "Managed-services contracts, ~25% annual logo churn"},
  "pipelines": {
    "_rule": "pipeline chosen by lead_source",
    "Acme MSP - Inbound": {"lead_sources": ["Google Ads", "Meta Ads", "Call Tracking"]},
    "Acme MSP - Outbound": {"lead_sources": ["Outbound"]},
    "Acme MSP - Referral": {"lead_sources": ["Referral"]},
  },
  "territories": {
    "NY": {"states": ["NY"], "owner": "Dana Whitfield"},
    "NJ": {"states": ["NJ", "PA"], "owner": "Marcus Lee"},
    "MA": {"states": ["MA", "CT", "RI"], "owner": "Priya Raman"},
    "TX": {"states": ["TX"], "owner": "Jordan Ellis"},
    "CA": {"states": ["CA"], "owner": "Sofia Alvarez"},
    "IL": {"states": ["IL"], "owner": "Owen Brooks"},
    "FL": {"states": ["FL"], "owner": "Grace Kim"},
  },
  "icp": {"industries": ["Healthcare", "Legal", "Financial Services", "Professional Services", "Manufacturing"],
    "employees_min": 20, "employees_max": 500}
}
```

8 GTM Engineer onboarding

One GTM Engineer can own this system across the portfolio. The role sits between RevOps, marketing ops and data: they keep the standard intact, keep the data clean, and roll the system out company by company.

Hiring profile

- 3+ years in RevOps, marketing ops or growth engineering
- HubSpot admin depth: properties, pipelines, workflows, imports, private apps
- SQL and Python fluent enough to own `gtm/model.py`
- Has built at least one webhook or API integration
- Can explain CAC payback to a CFO and a lead-routing bug to a rep

Interview exercise

Give the candidate this repo and one company's messy export. Ask for the scorecard in 3 hours and a list of the data problems found. Score on definitions held, problems surfaced and clarity of the write-up.

First 90 days

Days 1–30: own it. Run the pipeline end to end; reproduce every scorecard number by hand for one company; rotate the HubSpot token; fix the top three data-completeness gaps.

Days 31–60: make it live. Agent on live webhooks for both companies; scheduled daily run with Sheets sync; first monthly scorecard review with each company's GTM lead.

Days 61–90: propagate. Onboard a third portfolio company using Section 7; replace mock enrichment with a real provider; propose the first budget shift backed by linear attribution.

Operating cadence

CADENCE	WHAT	OUTPUT
Daily	Scheduled run; check agent errors and unassigned leads	Refreshed dashboard; Slack summary of failures
Weekly	Speed to lead, missed calls and SLA breaches by territory	Note to sales managers
Monthly	Scorecard review per company; data-completeness fixes	Board-ready scorecard; change log
Quarterly	Revisit benchmarks, scoring weights and lifetime assumptions	Updated <code>standard.json</code> with rationale

Ownership checklist

- ☐ Can run `python run_all.py` and explain each output file
- ☐ Knows where every metric is defined and why opportunities use `stage_reached`
- ☐ Has admin on HubSpot and owns the private app and its token
- ☐ Has added and removed a territory in config and seen routing change
- ☐ Has replayed leads through the agent and read the Slack and CRM output

- ☐ Owns the Looker Studio report and its Sheet
- ☐ Has onboarded a company end to end using Section 7

A Runbook and assumptions

Commands

```
pip install -r requirements.txt
python run_all.py                # raw data → cleaning → agent replay → model → HubSpot files → dashboard
python run_all.py --skip-generate # same, on raw exports already in data/<company>/raw/
python -m tests.test_pipeline    # cleaning round trip, attribution, agent fallback
python -m gtm.agent_server       # webhook server on :5000
python -m gtm.hubspot setup --apply # HubSpot properties + pipelines (HUBSPOT_TOKEN)
python -m gtm.sheets_sync        # push outputs to Google Sheets for Looker Studio
python -m playbook.build         # rebuild this PDF
```

Troubleshooting

SYMPTOM	LIKELY CAUSE	FIX
Lead routed to the unassigned queue	State missing or outside every territory	Add the state to a territory in config, or fix the form field
Deal creation fails in HubSpot	Pipeline label in config doesn't match HubSpot	Re-run <code>gtm.hubspot setup --apply</code> ; labels must match exactly
Webhook returns 401	Wrong client secret or clock skew over 5 minutes	Check <code>HUBSPOT_CLIENT_SECRET</code> and server time
CAC jumps month over month	Spend booked in a month with few closes	Read the 6-month blended figure; monthly CAC is noisy at low volume

Assumptions and limits

- All companies, people and records are synthetic. Generator inputs match the design targets; realized rates differ slightly because deals still open at the cut-off are not yet won or lost.
- Revenue is closed-won first-year contract value. LTV uses $ACV \times \text{margin} \times \text{lifetime}$; it ignores expansion and discounting.
- Enrichment is mocked. HubSpot, Slack, DeepSeek and Google Sheets clients are written against their public APIs; in the POC they were exercised against mocks (DeepSeek against a stand-in server that checks the request shape and JSON parsing).
- Everything runs on free tiers (HubSpot free CRM, Looker Studio, Google Sheets); DeepSeek is the only paid call.
- In production, add retries and a dead-letter queue to the agent, store secrets in a secrets manager, and move SQLite to the portfolio warehouse.